

TECHNICAL ARCHITECTURE DOCUMENT

Axia Data Flow Architecture

Complete mapping of user journeys, data flows,
component connections, and disconnection gaps
across the entire Axia platform.

Version 1.0 · June 2026 · Confidential

Table of Contents

1. Executive Summary

2. User Journey Map

2.1 Public Journey (Unauthenticated)

2.2 Authenticated User Journey

2.3 Client Portal Journey

3. Data Model Relationships

3.1 Core Entity Map

3.2 Workspace-Centric Data Hierarchy

4. Page-by-Page Data Flow Audit

4.1 Pages With Full Data Connectivity

4.2 Pages With Partial Connectivity

4.3 Pages With No Data / Blank States

5. Cross-Page Data Dependencies

6. Disconnection Map: What's Missing

6.1 Backend Queries With No Frontend Page

6.2 Frontend Pages Missing Data Connections

6.3 Critical Missing Routes

7. Recommendations: Connecting the Dots

7.1 Priority 1: Missing Pages (End-to-End Gaps)

7.2 Priority 2: Data Bridge Connections

7.3 Priority 3: Unused Backend Surface

8. Architecture Diagram

1. Executive Summary

Axia is a payment protection platform for agencies and freelancers working on platforms like Upwork, Fiverr, and Toptal. The application is built on Vite + React 19 with a Convex backend, comprising 33 page components, approximately 48 query calls, and 68 mutation calls across the frontend. The Convex backend exports roughly 150+ query functions, 80+ mutation functions, and 1 action function across 60+ module files.

KEY FINDING

40-50% of backend queries are never called from any frontend page. The backend has been "forward-built" with extensive functionality (compliance alerts, protection plans, milestone tracking, evidence analytics, network connections) that has no corresponding UI. Conversely, a few pages (Projects, InvoiceBuilder, ProposalBuilder) render sparse or empty content when the backend is unavailable, lacking proper demo data fallbacks.

This document maps the complete data flow architecture: how users navigate the platform, which data flows between pages, which sub-components consume which queries, and where the critical gaps exist. The goal is to provide a clear roadmap for connecting the remaining dots so that every piece of backend functionality has a visible, usable frontend surface.

33 Pages

Total page components in the application, from Landing to OwnerDashboard.

~48 Queries

Active useQuery calls across all pages and shared components.

~68 Mutations

Active useMutation calls for creating, updating, and deleting data.

2. User Journey Map

2.1 Public Journey (Unauthenticated)

Unauthenticated visitors experience a streamlined funnel: they arrive on the Landing page, join the waitlist via the WaitlistForm, receive a referral code through the WaitlistSuccessModal, and can view their position on the WaitlistSuccess page. The only data interactions are waitlist mutations and referral stat queries. No dashboard or workspace data is accessible.

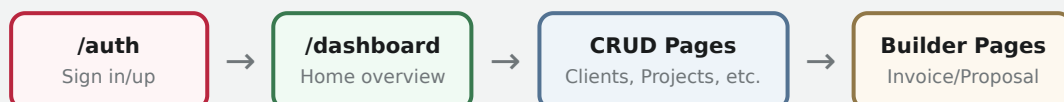


DATA FLOW

Landing page has zero data dependencies. WaitlistForm calls `addToWaitlist` mutation and `getEntryByEmail` query. SuccessModal calls `getReferralStats` query. The entire public journey is self-contained and works without authentication.

2.2 Authenticated User Journey

After authentication (via Google, GitHub, or email OTP on `/auth`), users enter the dashboard ecosystem. The `ProtectedRoute` component redirects unauthenticated users to `/auth`. The `WorkspaceProvider` wraps all routes, providing workspace context. The `DashboardLayout` provides a `CollapsibleSidebar` + main content area.



The authenticated journey branches into several functional areas, each with its own data domain:

FUNCTIONAL AREA	PAGES	PRIMARY DATA	CROSS-DEPENDENCIES
CRM	Clients, ClientWorkspace	clients table	Projects, Invoices, Proposals reference clientId
Project Protection	Projects, Scope, EvidenceLibrary	projects, scopeDefinitions	Clients (owner), WorkSessions, TimeBlocks
Pipeline & Deals	Pipeline	pipelineStages, deals	Clients (deal.clientId), Proposals (deal.proposalId)
Billing	Invoices, InvoiceBuilder, PaymentPatterns, Subscription	invoices, invoiceWorkLinks	Clients (invoice.clientId), WorkSessions
Proposals	Proposals, ProposalBuilder	proposals, proposalFollowUps	Clients, Deals (deal.proposalId)
Time Tracking	TimeTracking, EvidenceExport	workSessions, timeBlocks	Projects, EvidenceSessions
Team & Workspace	TeamManagement	workspaceMembers, teams	All workspace-scoped data
Communication	Messages	channels, messages	WorkspaceMembers (participants)
Analytics	Reports, Goals, Tags, Dashboard	disputeReports, goals, tags	WorkSessions, Projects, Invoices
Integrations	PlatformIntegrations	platformConnections	User account
Admin	OwnerDashboard	waitlistEntries (admin)	Stripe/Airtable external data

2.3 Client Portal Journey

Clients access the platform via a shared workspace token URL (/client-workspace?token=xxx). This route validates the token, then loads the client's projects, proposals, invoices, and team members. Clients can view proposals and invoices, mark them as viewed, and the system records their access. This is the only page that works with token-based authentication instead of user authentication.

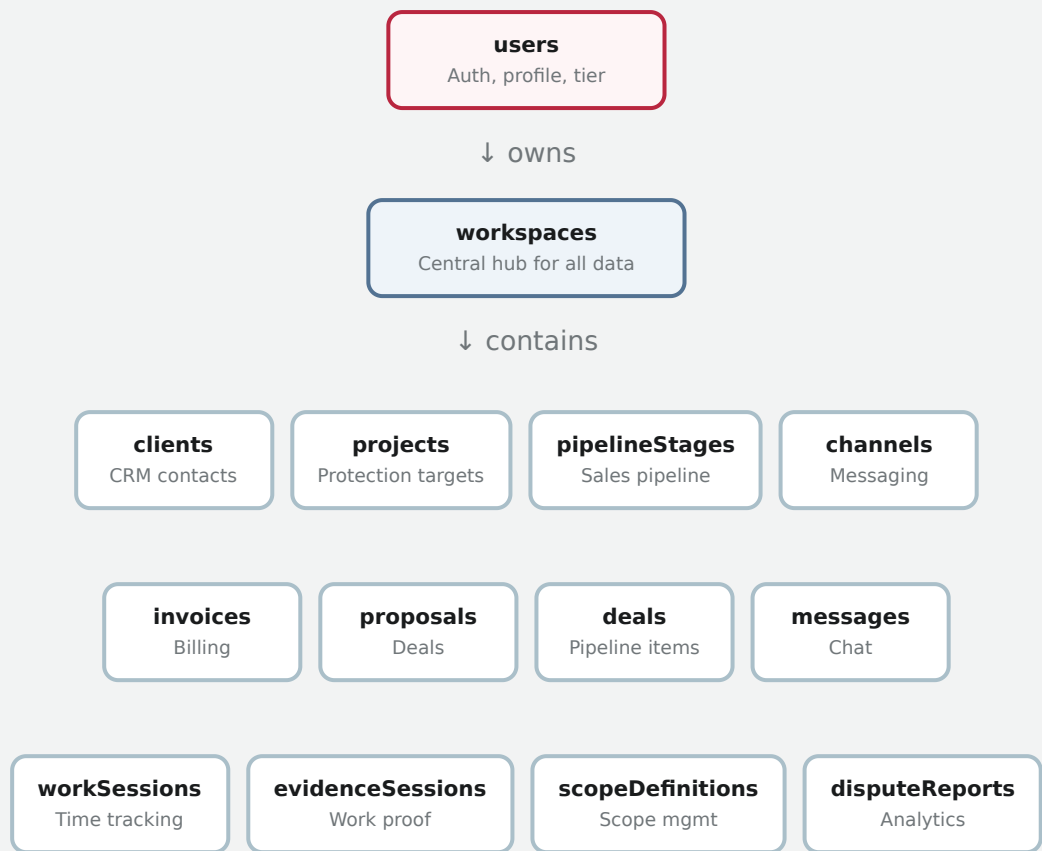
GAP IDENTIFIED

The client portal does NOT have pages for proposal signing or scope approval. The backend has `proposals.crud.getProposalByToken` and `scope.crud.getScopeByApprovalToken` queries, but no frontend routes exist for clients to view proposals or approve scopes via shared links.

3. Data Model Relationships

3.1 Core Entity Map

The Convex data model is workspace-centric: every major entity belongs to a workspace. Users can own or be members of multiple workspaces. Below is the primary entity relationship map showing how data flows from the user through to invoices and evidence.



3.2 Workspace-Centric Data Hierarchy

The workspace is the root container for nearly all business data. Each workspace has members, teams, channels, clients, and pipeline stages. Clients belong to a workspace and have projects, invoices, and deals. Projects have work sessions, evidence sessions, scope definitions, and protection scores. This hierarchy ensures data isolation between workspaces and provides the foundation for multi-tenant access control.

Parent Entity	Child Entities	Join Key	Pages That Use This Chain
workspaces	clients, projects, invoices, proposals, deals, pipelineStages, channels, goals, tags, scopeDefinitions, workSessions	workspaceId	Dashboard (8 queries), Clients, Invoices, Proposals, Pipeline, Scope, TimeTracking, TeamManagement, Messages
clients	projects, invoices, deals	clientId	Clients (enriched view), PaymentPatterns, ClientWorkspace
projects	workSessions, evidenceSessions, scopeDefinitions, milestoneSnapshots	projectId	Projects, EvidenceLibrary, Scope, ProtectionValueDashboard
invoices	invoiceWorkLinks, paymentReminders	invoiceId	Invoices, InvoiceBuilder, PaymentPatterns, Subscription
proposals	proposalFollowUps	proposalId	Proposals, ProposalBuilder
workSessions	timeBlocks, evidenceSessions, disputeReports	sessionId	TimeTracking, EvidenceLibrary, Reports
channels	channelMembers, messages	channelId	Messages
messages	reactions, mentions	messageId	Messages (thread view)

4. Page-by-Page Data Flow Audit

4.1 Pages With Full Data Connectivity **Connected**

These pages have both backend queries AND demo/mock data fallbacks. They render meaningful content whether the Convex backend is available or not.

PAGE	QUERIES USED	MUTATIONS USED	DEMO DATA
Dashboard	8 (clients, pipeline, proposals, invoices, scope, deals)	1 (autoSeed)	Zeroed stats + "Seed Demo Data" CTA
Clients	1 (getClients)	4 (CRUD + share/unshare)	MOCK_CLIENTS (5 items)
Invoices	2 (getInvoices, getInvoiceStats)	6 (CRUD + send + seed)	MOCK_INVOICES (6 items)
Proposals	3 (getProposals, getProposalStats, getFollowUps)	6 (CRUD + share + seed)	MOCK_PROPOSALS
Pipeline	3 (getStages, getDeals, getPipelineStats)	8 (CRUD + seed + import)	MOCK_STAGES + MOCK_DEALS
TeamManagement	5 (members, stats, invitations, teams, teamMembers)	9 (CRUD)	MOCK_MEMBERS + MOCK_TEAMS
Messages	4 (channels, messages, threads, members)	7 (CRUD + reactions + pins)	Extensive mock dataset
TimeTracking	2 (currentSession, getSessions)	6 (start/stop/pause/resume)	MOCK_ENTRIES (5 sessions)
EvidenceLibrary	2 (getLibraryData, getTimeline)	0	safeEvidenceData fallback
Reports	1 (getUserDisputeReports)	2 (create, updateStatus)	MOCK_REPORTS (5 items)
Tags	1 (getTags)	3 (CRUD)	MOCK_TAGS (10 items)
Goals	1 (getGoals)	5 (CRUD + milestone)	MOCK_GOALS (4 items)
PaymentPatterns	3 (getInvoices, getInvoiceStats, getClientsEnriched)	0	MOCK_CLIENTS + MOCK_INVOICES
OwnerDashboard	2 (getAllWaitlistEntries, getWaitlistCount)	0	Extensive MOCK_DATA
Scope	3 (getScopeDefinitions, getChangeOrders x2)	4 (CRUD + approve)	MOCK_SCOPES (3 items)
PlatformIntegrations	1 (getPlatformConnectionStatus)	3 (connect/disconnect)	Demo mode cards
Subscription	2 (getInvoices, getInvoiceStats)	0	Hardcoded tier definitions
HelpCenter	0	0	MOCK_ARTICLES + MOCK_TICKETS
ClientDashboard	1 (getClientProfile)	0	mockClientProfile
Landing	0	0	N/A (static marketing)
Auth	0	0	N/A (authentication flow)

4.2 Pages With Partial Connectivity **Partial**

These pages have queries but lack demo data fallbacks. When the Convex backend is unavailable, they render empty states or skeleton forms rather than meaningful content.

PAGE	QUERIES USED	MISSING FALLBACK	RESULT WHEN DISCONNECTED
Projects	1 (getMyProjects)	No mock project data	Empty list with "Add Test Project" button. No project cards rendered.
InvoiceBuilder	2 (getInvoice, getWorkLinks)	No mock invoice for editing	New invoice form works, but edit mode shows empty.
ProposalBuilder	3 (getProposal, getDeal, getTemplates)	No mock proposal for editing	New proposal form works, but edit mode shows empty.
WaitlistSuccess	1 (getReferralStats)	Only zeroed fallback	Shows position: 0, referrals: 0. Still functional but uninformative.
EvidenceExport	3 (getLibraryData, getClients, getScopeDefinitions)	Partial mock data	Mock evidence items available but client/scope selectors empty.

4.3 Pages With No Data / Blank States **Blank**

These pages either have no Convex queries or exist as pure forms/static content. They are functional but represent "dead ends" in the data flow where no data enters or leaves.

PAGE	DATA STATUS	NOTES
OnboardingSource	No queries, no mutations	Stub completeOnboarding function. Saves to localStorage only.
OnboardingUserInformation	No queries, no mutations	Pure form. Saves to localStorage only.
ClientSignup	1 mutation only	Register mutation saves to Convex but form is purely local.
ClientLogin	1 query (unused in render)	Login always succeeds in demo mode via localStorage.
NotFound	No data	Static 404 page.
ApiSettings	No Convex queries	Entirely mock API keys and webhooks (local state only).

5. Cross-Page Data Dependencies

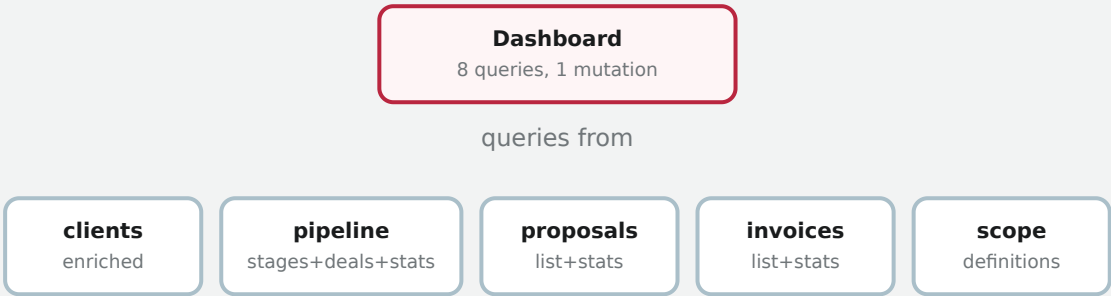
Several data entities are queried from multiple pages simultaneously. This creates shared dependencies where the same data must be consistent across views. The WorkspaceProvider acts as a shared context, but individual pages query the same data independently (Convex handles deduplication automatically).

SHARED DATA	PAGES THAT QUERY IT	CONSISTENCY RISK
clients (getClients / getClientsEnriched)	Dashboard, Clients, PaymentPatterns, EvidenceExport	Low - Convex real-time sync ensures consistency
invoices (getInvoices / getInvoiceStats)	Dashboard, Invoices, PaymentPatterns, Subscription, InvoiceBuilder	Low - same query function, Convex deduplicates
proposals (getProposals / getProposalStats)	Dashboard, Proposals, ProposalBuilder	Low - real-time subscription model
pipeline (getStages / getDeals / getPipelineStats)	Dashboard, Pipeline	Low - same workspace scope
scopeDefinitions	Dashboard, Scope, EvidenceExport	Low - read-only in Dashboard/EvidenceExport
workspaceId (from WorkspaceProvider)	All workspace-scoped pages	HIGH - If workspaceId is invalid/null, all workspace queries skip

CRITICAL DEPENDENCY: WORKSPACEID

The workspaceId from WorkspaceProvider is the linchpin of the entire data architecture. Nearly every page passes workspaceId to its queries. If the user has no workspace (e.g., first login before auto-seed completes), ALL workspace-scoped queries return undefined, and pages show empty/demo states. The WorkspaceProvider attempts to auto-seed a personal workspace, but if this mutation fails or is slow, pages render with no data.

Below is a diagram showing how the Dashboard page serves as the primary data aggregation point, querying 8 different data sources simultaneously to build its overview:



6. Disconnection Map: What's Missing

6.1 Backend Queries With No Frontend Page

The Convex backend has approximately 50+ query functions that are never called from any front-end page. These represent "forward-built" features that exist in the backend but have no UI surface. Grouped by functional area:

FUNCTIONAL AREA	UNUSED QUERIES	IMPACT
Compliance & Alerts	complianceAlerts.getActiveAlerts, platforms.upworkComplianceCheck.getUpworkCompliance	No way for users to see compliance issues or Upwork policy violations
Evidence Analytics	evidence.analytics.getEvidenceAnalytics	No analytics dashboard for evidence quality or coverage
Protection Plans (Premium)	premium.protectionPlans.getProtectionPlans, premium.crossPlatformVerification, premium.teamValidation, premium.protectionAdvisor	4 premium features with no UI - major monetization gap
Milestone Tracking	milestoneSnapshots.getSnapshots, milestoneAlerts.getAlerts, milestoneReports.getReports	3 milestone features with no browsing/management UI
Scope Formalization	scopeFormalization.getFormalizations	Formalizations can be created but never listed
Client Intelligence	clients.getStats, clients.search, clients.getById, timeBlocks.calculateRejectedHours	No client search, stats overview, or rejected hours calculation in UI
Payment Reminders	billing.reminders.getReminderHistory, billing.reminders.getAllReminders	Reminder history exists but no page shows it
Network	network.premiumNetwork.getNetworkConnections	PremiumNetwork component exists but no dedicated page
Tier Detection	tiers.tierDetection.detectTier	Tier detection logic exists but isn't surfaced in Subscription page
Client Policies	policies.clientPolicies.getClientPolicies	Policy management UI missing
WCVAM Verification	wcvm.contextScanner.getVerificationData	Work Context Verification Model has no dedicated UI

6.2 Frontend Pages Missing Data Connections

Some pages exist but don't connect to available backend data that would enhance them:

PAGE	AVAILABLE BACKEND DATA NOT CONNECTED	WHAT'S MISSING
Projects	projectHealthDashboard, projectProtectionScore, riskTimeline, adaptiveEvidenceSystem	Project detail view with health dashboard, risk timeline, and protection score
Subscription	tiers.tierDetection.detectTier	Auto-detection of user's optimal tier based on usage
TeamManagement	workspaces.members.searchMembers, workspaces.members.getMemberProjects	Member search and project assignment views
Dashboard	complianceAlerts.getActiveAlerts, evidence.analytics	Compliance alerts widget and evidence quality summary
Invoices	billing.reminders.getReminderHistory	Reminder history per invoice
TimeTracking	timeBlocks.getSessionBlocks, timeBlocks.calculateRejectedHours	Block-level detail view and rejected hours calculation

6.3 Critical Missing Routes

These are entire pages that should exist but don't. They represent end-to-end gaps where the backend functionality exists but there's no way for users to access it:

CRITICAL GAP 1: PROPOSAL CLIENT VIEW

Backend exists: `proposals.crud.getProposalByToken` returns a proposal by its public sharing token. `proposals.crud.signProposal` and `proposals.crud.markProposalViewed` handle client actions.

Frontend missing: No `/proposal/view/[token]` route exists. When a client receives a proposal link, they have no page to view or sign it. The entire proposal sharing workflow is broken at the receiving end.

CRITICAL GAP 2: SCOPE CLIENT APPROVAL

Backend exists: `scope.crud.getScopeByApprovalToken` and `scope.crud.approveScopeByClient` handle client-facing scope approval.

Frontend missing: No `/scope/approve/[token]` route. Clients cannot review or approve scope changes through the platform. This is a core value proposition feature that is entirely non-functional.

CRITICAL GAP 3: PROJECT DETAIL PAGE

Backend exists: `projectProtection.getProjectProtectionDetails`, `projectHealthDashboard`, `projectRiskTimeline`, `adaptiveEvidenceSystem`, `milestoneProtection` - all return rich project data.

Frontend missing: The Projects page only shows a flat list. Clicking a project should navigate to a detail page showing health dashboard, risk timeline, milestone tracking, and protection score. None of this exists.

GAP 4: INVOICE CLIENT VIEW

Backend exists: `billing.crud.getByPublicToken` returns an invoice by sharing token.

Frontend missing: No `/invoice/view/[token]` route for clients to view invoices externally. The ClientWorkspace page shows invoices, but direct link sharing doesn't work.

7. Recommendations: Connecting the Dots

7.1 Priority 1: Missing Pages (End-to-End Gaps)

These are the highest-priority items because they represent broken user journeys. The backend is ready, the sharing infrastructure exists, but there's no page for the recipient to interact with:

#	ROUTE TO CREATE	BACKEND QUERIES TO WIRE	ESTIMATED EFFORT	BUSINESS IMPACT
1	/proposal/view/:token	getProposalByToken, signProposal, markProposalViewed	Medium (2-3 days)	Critical - proposal sharing is a core feature
2	/scope/approve/:token	getScopeByApprovalToken, approveScopeByClient	Medium (2-3 days)	Critical - scope approval is a core value prop
3	/projects/:id (Project Detail)	getProjectProtectionDetails, getProjectHealthDashboard, getProjectRiskTimeline, getMilestoneProtection, getAdaptiveEvidenceSystem	Large (5-7 days)	High - Projects page is currently a flat list
4	/invoice/view/:token	getByPublicToken	Small (1 day)	Medium - client invoice viewing

7.2 Priority 2: Data Bridge Connections

These are connections between existing pages and existing backend queries that would enhance the user experience without requiring new pages:

PAGE	ENHANCEMENT	QUERY TO ADD	EFFORT
Dashboard	Add compliance alerts widget	complianceAlerts.getActiveAlerts	Small
Dashboard	Add evidence quality summary	evidence.analytics.getEvidenceAnalytics	Small
Subscription	Auto-detect optimal tier	tiers.tierDetection.detectTier	Small
Projects	Add mock projects for demo mode	N/A (frontend mock data)	Small
InvoiceBuilder/ProposalBuilder	Add mock data for edit mode demo	N/A (frontend mock data)	Small
TeamManagement	Add member search when assigning	workspaces.members.searchMembers	Small
TimeTracking	Show rejected hours calculation	timeBlocks.calculateRejectedHours	Small
Invoices	Show reminder history per invoice	billing.reminders.getReminderHistory	Small

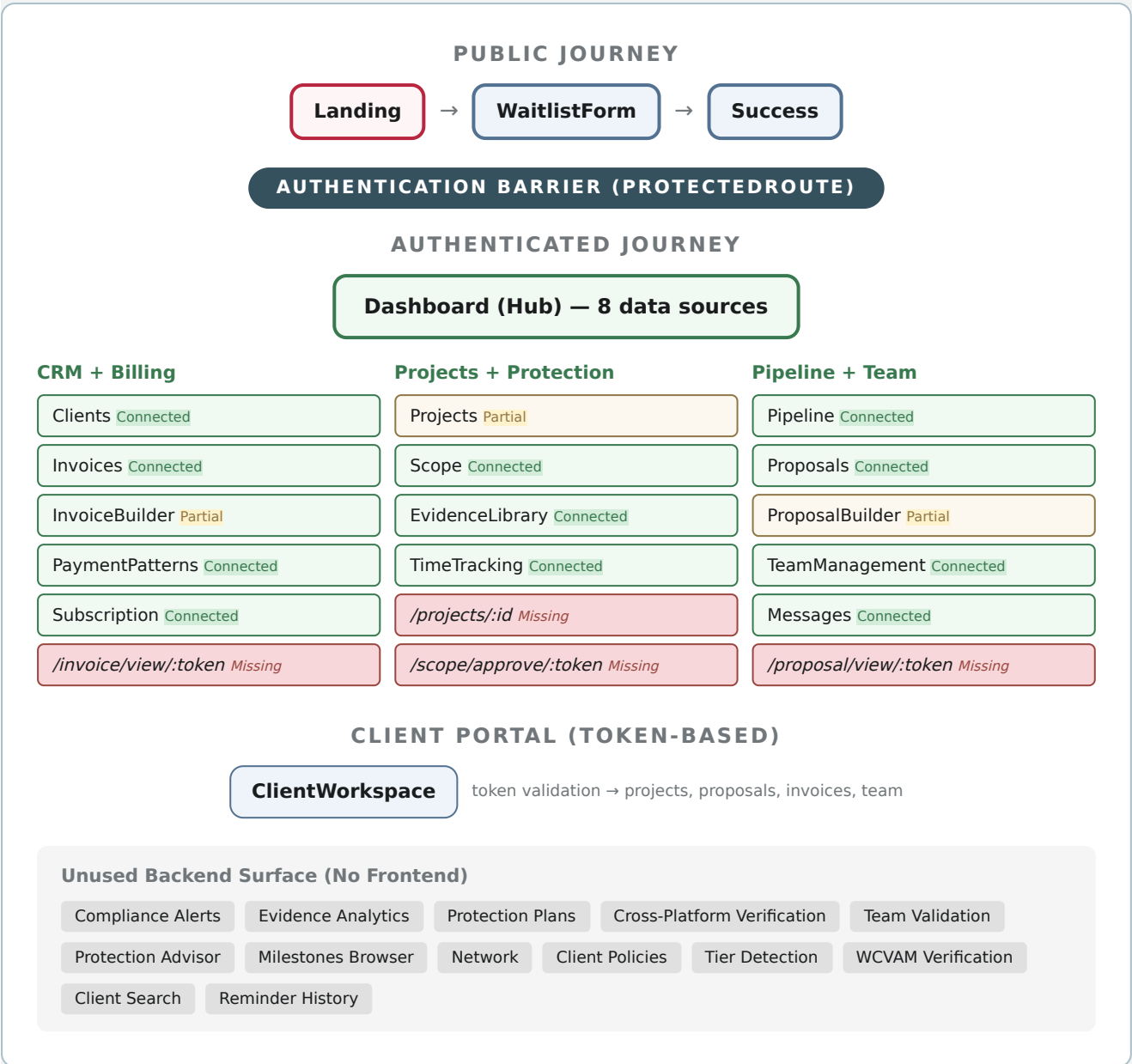
7.3 Priority 3: Unused Backend Surface

These are features that have full backend implementations but no frontend. They represent potential product features that could be unlocked with UI development:

FEATURE AREA	BACKEND QUERIES AVAILABLE	SUGGESTED PAGE	BUSINESS VALUE
Protection Plans (Premium)	getProtectionPlans, createProtectionPlan	New "Protection Plans" page under billing	Monetization - premium feature gating
Cross-Platform Verification	getCrossPlatformVerification	Enhancement to PlatformIntegrations	Core value prop - multi-platform evidence
Team Validation	getTeamValidations	Enhancement to TeamManagement	Premium - team accountability
Protection Advisor	getAdvisorAlerts	New "Advisor" page or Dashboard widget	Proactive protection recommendations
Milestone Browser	getSnapshots, getAlerts, getReports	Tab within Project Detail page	Project tracking and documentation
Network / Freelancer Directory	getNetworkConnections, getVerifiedFreelancers	New "Network" page	Community and trust network
Client Policies	getClientPolicies, createClientPolicy	Tab within Clients detail view	Platform-specific policy management
Compliance Dashboard	getActiveAlerts, getUpworkCompliance	New "Compliance" page	Risk prevention - core value prop

8. Architecture Diagram

The diagram below shows the complete user journey from landing to each functional area, with data flow connections indicated by arrows. Green nodes are fully connected, amber nodes are partially connected, and red nodes represent missing pages that need to be built.



SUMMARY

Axia has a robust backend with 150+ query functions and a well-structured workspace-centric data model. The frontend covers approximately 50-60% of this surface area with 33 pages, most of which are fully connected with demo data fallbacks. The three critical gaps are: (1) Proposal client view page, (2) Scope client approval page, and (3) Project detail page. Addressing these would close the most impactful end-to-end gaps. Beyond that, 11 backend feature areas have no frontend surface at all, representing significant untapped product potential.